PART 2A: The Front End & Presentation Layer

Complete Technical Architecture, Full-Stack SCADA Implementation & Detailed Code Dissection

Author: Saptarshi Das (VIT Vellore)
Email: saptarshid234@outlook.com
Repository: saptarshidas578/smartgrid_ai

Primary Modules: src/dashboard/grid_dashboard.py
Services: grid_analytics.py, telegram_service.py
Role: Web SCADA, WebSockets, Reporting & Alerts

1. Executive Front-End Architecture & Design Philosophy

Part 2A constitutes the human-facing operational tier of the smart grid. It provides plant supervisors and electrical engineers with complete situational awareness, telemetry visualization, manual control overrides, and automated reporting. Built on an asynchronous **FastAPI** web server, the presentation tier pushes real-time telemetry over WebSockets at 1.0 Hz, eliminating polling latency while guaranteeing that UI operations remain strictly non-blocking to the underlying power safety loop.

Component	Underlying Stack	Protocol / Channel	Operator Capability
SCADA Web UI	FastAPI + HTML5/CSS3/JS	HTTP (Port 8000) / WebSockets	Live telemetry, dynamic gauges, status badges, manual relay switches
WebSocket Stream	Asyncio WebSocket Server	/ws/telemetry (1 Hz)	Zero-refresh live streaming of currents, voltages, powers, and alarms
Analytics Engine	Matplotlib + Pandas	On-Demand REST API	Generates energy pie charts, daily load profiles, voltage stability curves
Executive Reporter	ReportLab Platypus Engine	Downloadable PDF	Compiles IEEE reliability indices (SAIFI/SAIDI), peak demands, and costs
Mobile Alerter	Telegram Bot API (httpx)	HTTPS Webhooks (Port 443)	Dispatches push alerts for source failure, 2-sigma trips, and stuck contactors
CLI Console UI	Terminal VT100 / ANSI	Standard Output (stdout)	ASCII capacity bar, system uptime, and remote tunnel link URLs

2. Detailed Line-by-Line Dissection: src/dashboard/grid_dashboard.py

Section 2.1: FastAPI App Initialization & Global Shared State (Lines 1–65)

```
39: app = FastAPI(title="Industrial Smart Grid SCADA & Digital Twin")
41: # Global Shared State Dictionary
42: latest_telemetry: Dict[str, Any] = {
43:     "timestamp": datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
44:     "esp32_online": False,
45:     "hardware_status": "OFFLINE",
47:     "mode": "DUAL SMPS",
48:     "current_limit": 5000.0,
49:     "total_current": 0.0,
50:     "total_power": 0.0,
51:     "predicted_current": 0.0,
52:     "predicted_power": 0.0,
55:     "control_mode": "AUTO",
60:     "sources": {"smps1": False, "smps2": False, "battery": False},
61:     "critical": {"voltage": 12.0, "current": 300.0, "power": 3600.0},
62:     "hospital": {"voltage": 12.0, "current": 0.0, "power": 0.0, "relay": 1, "priority": 4},
63:     "street":   {"voltage": 12.0, "current": 0.0, "power": 0.0, "relay": 1, "priority": 3},
64:     "house":    {"voltage": 12.0, "current": 0.0, "power": 0.0, "relay": 1, "priority": 2},
65:     "industry": {"voltage": 12.0, "current": 0.0, "power": 0.0, "relay": 1, "priority": 1}
66: }
```

Lines 39–40: Initializes the core FastAPI application instance with title and OpenAPI metadata.

Lines 42–66: Defines the thread-safe global telemetry state dictionary. It mirrors the exact nested JSON schema emitted by the ESP32 hardware and updated by the backend MQTT loop at 1.0 Hz.

Section 2.2: Thread-Safe WebSocket Connection Pooling (Lines 100–140)

```
102: class ConnectionManager:
103:     def __init__(self):
104:         self.active_connections: List[WebSocket] = []
106:     async def connect(self, websocket: WebSocket):
107:         await websocket.accept()
108:         self.active_connections.append(websocket)
110:     def disconnect(self, websocket: WebSocket):
111:         if websocket in self.active_connections:
112:             self.active_connections.remove(websocket)
114:     async def broadcast(self, message: dict):
115:         for connection in list(self.active_connections):
116:             try:
117:                 await connection.send_json(message)
118:             except Exception:
119:                 self.disconnect(connection)
122: manager = ConnectionManager()
```

Lines 102–108: Implements active WebSocket connection pooling. Accepts incoming client upgrade handshakes and registers browser client sockets.

Lines 114–119 (Resilient Broadcasting): Broadcasts telemetry JSON packets to all connected operator screens. If a browser tab is abruptly closed or network drops, it gracefully purges the stale socket without throwing unhandled exceptions.

Section 2.3: REST Actuation Control & Safety Dwell Interception (Lines 170–215)

```
180: @app.post("/api/control/relay")
181: async def control_relay(request: Request):
182:     data = await request.json()
183:     load = data.get("load", "").lower()
184:     state = int(data.get("state", 1))
185:     # Enforce Dwell-Time Safety Lockout
186:     allowed, reason = safety_engine.validate_manual_switch(load, state)
187:     if not allowed:
188:         raise HTTPException(status_code=429, detail=reason)
189:     # Publish to MQTT Actuator Topic
190:     payload = {"cmd": "MANUAL", "load": load, "state": state, "reason": "Operator Override"}
191:     mqtt_client.publish(config.mqtt_topic_control, json.dumps(payload), qos=1)
192:     return {"status": "SUCCESS", "load": load, "state": state}
195: @app.post("/api/control/mode")
196: async def set_control_mode(request: Request):
197:     data = await request.json()
198:     mode = data.get("mode", "AUTO").upper()
199:     safety_engine.control_mode = mode
200:     return {"status": "SUCCESS", "control_mode": mode}
```

Lines 180–188: Human operator relay actuation endpoint. **Crucial Safety Guard:** Human commands are validated against safety_engine.validate_manual_switch(). If an operator tries to toggle a relay before the 15s dwell time expires, it rejects the request with HTTP 429.

Lines 189–192: Transmits the validated actuation payload to HiveMQ over MQTT QoS 1.

Lines 195–200: Allows the operator to toggle between AUTO (Safety Governor controls relays) and MANUAL mode.

Section 2.4: 1 Hz Cyclic WebSocket Push Loop & Client Endpoint (Lines 220–250)

```
220: async def telemetry_broadcaster():
221:     while True:
222:         if manager.active_connections:
223:             await manager.broadcast(latest_telemetry)
224:         await asyncio.sleep(config.update_interval_sec)  # 1.0s interval
227: @app.websocket("/ws/telemetry")
228: async def websocket_endpoint(websocket: WebSocket):
229:     await manager.connect(websocket)
230:     try:
231:         while True:
232:             _ = await websocket.receive_text()  # Keep-alive heartbeat
233:     except WebSocketDisconnect:
234:         manager.disconnect(websocket)
```

Lines 220–224: Background async worker looping continuously, broadcasting the latest telemetry state every second.

Lines 227–234: WebSocket endpoint maintaining persistent full-duplex communication with connected browser clients.

3. Detailed Dissection: src/services/grid_analytics.py

Section 3.1: Energy Integration, IEEE Reliability & Financial Billing

```
61: def compute_analytics(df_tel: pd.DataFrame, df_act: pd.DataFrame, tariff: float = DEFAULT_TARIFF_PER_KWH) -> dict:
75:     # 1. Numerical kWh Integration via Trapezoidal Riemann Rule
76:     dt_hours = 2.0 / 3600.0  # 2-second sampling interval in hours
77:     kwh_hospital = (df_tel["hospital_power"].sum() / 1000.0) * dt_hours
78:     kwh_total    = (df_tel["total_power"].sum() / 1000.0) * dt_hours
82:     # 2. Financial Billing Integration (INR ■)
83:     total_cost_inr = kwh_total * tariff
90:     # 3. IEEE Standard Grid Reliability Indices
91:     # SAIFI = Total Customer Interruptions / Total Customers
92:     # SAIDI = Total Interruption Durations / Total Customers
93:     saifi = float(len(df_act)) / 4.0
94:     saidi_minutes = (float(len(df_act)) * 15.0) / 60.0
```

Lines 75–78 (Numerical Energy Integration): Converts discrete power readings into cumulative energy consumption using the Riemann sum: $E = \sum P_k \cdot \Delta t$, dividing by 1000 to output kilowatt-hours (kWh).

Lines 82–83: Computes commercial power bills using standard industrial tariff rates (■8.00 / kWh).

Lines 90–94: Formulates standard IEEE power distribution reliability indices: SAIFI (frequency of interruptions) and SAIDI (duration of outages).

Section 3.2: Matplotlib Visual Chart Generation Engine

```
120: def generate_analytics_charts(df_tel: pd.DataFrame, results_dir: str):
125:     # 1. Sector Energy Consumption Pie Chart
126:     fig, ax = plt.subplots(figsize=(6, 5), dpi=300)
127:     sectors = ["Hospital", "Street", "House", "Industry", "Critical"]
128:     powers = [df_tel[f"{s.lower()}_power"].sum() for s in sectors]
129:     ax.pie(powers, labels=sectors, autopct='%1.1f%%', colors=['#2563eb', '#f59e0b', '#10b981', '#8b5cf6', '#dc2626'])
130:     fig.savefig(os.path.join(results_dir, "chart_energy_pie.png"), bbox_inches='tight')
140:     # 2. Voltage Stability Curve
141:     fig, ax = plt.subplots(figsize=(8, 4), dpi=300)
142:     ax.plot(df_tel["timestamp"], df_tel["critical_voltage"], color='#2563eb', label='Bus Voltage (V)')
143:     ax.axhline(12.0, color='gray', linestyle=':')
144:     fig.savefig(os.path.join(results_dir, "chart_voltage_stability.png"), bbox_inches='tight')
```

Lines 125–130: Generates high-resolution 300 DPI sector energy consumption distribution charts.

Lines 140–144: Visualizes bus voltage tracking against nominal 12.0 V boundaries across source transitions.

4. Detailed Dissection: src/services/telegram_service.py

Section 4.1: Asynchronous Message Queue & Mobile Push Alerter

```
28:         self._queue = queue.Queue(maxsize=250)
29:         self._worker_thread = threading.Thread(target=self._worker_loop, daemon=True)
30:         self._worker_thread.start()
32:     def _worker_loop(self):
33:         while True:
34:             html_msg = self._queue.get()
36:             self._post_telegram_sync(html_msg)
37:             self._queue.task_done()
38:             time.sleep(0.3)  # Telegram API rate limit throttle
66:     def alert_actuation_event(self, action: str, load: str, reason: str, curr: float, limit: float):
75:         msg = f"■ <b>SMART GRID ACTION: {action}</b>\n" \
76:               f"Feeder: <b>{load.upper()}</b>\n" \
77:               f"Trigger Reason: <code>{reason}</code>\n" \
78:               f"Load at Event: <b>{curr:.1f} mA</b> / Limit: <b>{limit:.0f} mA</b>"
79:         self.dispatch_async(msg)
```

Lines 28–38: Decouples notification dispatch onto a dedicated thread with a 250-item FIFO queue, ensuring external network latency never blocks the 1 Hz telemetry control loop.

Lines 66–79: Formats structured mobile alert notifications and dispatches them asynchronously to the operator's phone.

5. Detailed Dissection: src/smart_grid_ai.py (Terminal SCADA)

Section 5.1: Low-Overhead Terminal ASCII Dashboard (print_ascii_dashboard)

```
97: def print_ascii_dashboard(row: dict, in_startup: bool, startup_rem: float, action_taken: str):
98:     limit = float(row.get("current_limit", 5000.0))
99:     tot_curr = float(row.get("total_current", 0.0))
100:     utilization = (tot_curr / limit * 100.0) if limit > 0 else 0.0
127:     # Dynamic Capacity Meter Bar
128:     bar_len = 35
129:     filled = int(min(utilization, 100.0) / 100.0 * bar_len)
130:     bar = "■" * filled + "■" * (bar_len - filled)
131:     print(f" CAPACITY   : [{bar}] {utilization:5.1f}%")
132:     print(f" TOTAL LOAD : {tot_curr:6.1f} mA / {limit:.0f} mA | PRED: {row.get('predicted_current', 0.0):6.1f} mA")
```

Lines 97–100: Calculates real-time source capacity utilization percentage.

Lines 127–132: Renders an ASCII status bar directly onto the server console, providing headless status monitoring without requiring a graphical display.